🪞 Autoevaluación

Tabla de contenidos

1 Variables 1
2 Números 2
3 Booleanos 2
4 Cadenas de caracteres 2
5 Funciones 3
6 Listas 3
7 Tuplas 5
8 Diccionarios 5
9 Control de flujo 6
9.1 Condicionales 6
9.2 Bucles 6

1 Variables

1. ¿Qué valor contiene la variable `contador` luego de ejecutar el siguiente código?

```
contador = 20
contador + 1
```

2. Complete la siguiente tabla indicando si el nombre es válido para una variable de Python.

Nombre	¿Válido?	Justificación
`balance`		
`balanceActual`		
`balance-actual`		
`balance actual`		
`balance_actual`		
`mis.datos`		
`0cuenta`		
`cuenta0`		
`_cuenta`		
`SPAM`		
`cantidad$`		

2 Números

1. ¿Cuál es el tipo de `10 / 2`? ¿Por qué?
2. ¿Cuál es el tipo de `5 * 2`? ¿Por qué?
3. ¿Por qué `5 == 5.0` es `True`?

3 Booleanos

1. ¿Cuál es el resultado de las siguientes expresiones? ¿Por qué?

```
False or not False
```

```
not (True and True)
```

```
not True and True
```

```
"True" != True
```

```
10 > 5 + 3
```

```
None is None
```

```
False is False
```

2. Ejecute línea por línea los siguientes bloques y analice los resultados.

```
int(True) * 50
True * 50
```

```
1 is True
bool(1) is True
id(True)
id(bool(1))
id(bool(1024))
```

3. ¿Cuáles son los 3 operadores booleanos?

4 Cadenas de caracteres

1. ¿Por qué la siguiente comparación resulta en `True`?

```
"spam" + "spamspam" == "spam" * 3
```

2. ¿Por qué la siguiente expresión resulta en un error? ¿Cómo se puede arreglar?

```
"Me comí " + 6 + " panchos."
```

3. ¿Encuentra algo extraño en la siguiente expresión? ¿Cómo la mejoraría?

```
materia = "Programación 2"
print("¡Sean bienvenidos a la materia {materia}!")
```

4. Explique por qué es redundante utilizar `str()` en el siguiente bloque de código:

```
x, y = 10, 20
print(f"La suma de {str(x)} y {str(y)} es: {str(x + y)}.")
```

5. Considere el siguiente bloque de código:

```
mensaje = "Hola, ¿cómo estes?"
mensaje[-3] = "á"
```

   - ¿Cuál es la intención detrás del programa?
   - ¿Por qué no funciona?
   - ¿Cómo podría arreglarlo? Alerta: la solución no es muy elegante.

6. ¿Cuál es el resultado de `list("abcdefgh")`? ¿Por qué?

7. ¿Por qué `set("abcde")` es distinto de `{"abcde"}`?

5 Funciones

1. ¿Cuál es la diferencia entre una función y una llamada a función?

2. ¿Cuál es el valor que devuelve una función que no tiene `return`?

3. ¿Cuándo se ejecuta el código dentro de una función: cuando la función se define o cuando se la llama? Considere la siguiente función:

```
def suma(x, y):
    print(100 / 0)
    return x + y
```

   Luego, ejecute el siguiente bloque:

```
suma(2, 4)
```

6 Listas

1. ¿Qué es `[]`? ¿Cuál es el resultado de `len([])`?

2. ¿Por qué la siguiente expresión resulta en `False`? Ayuda: use la función `id()`.

```
[] is []
```

3. ¿Por qué se obtiene un error en el siguiente bloque de código?

```
l = []
l[0]
```

4. Si `[0][0]` devuelve `0`, ¿por qué `[1][1]` no devuelve `[1]`?
5. ¿Cuál es el valor de `x` en el siguiente bloque de código? ¿Por qué?

```
l = ["hola", "hola hola", "hasta luego"]
x = l.remove("hola")
```

6. ¿Cómo le asignaría el valor `"hola"` como el tercer valor en una lista almacenada en una variable llamada `cosas`? Asuma que `cosas` contiene `[2, 4, 6, 8, 10]`.
7. Asuma que `letras` contiene la lista `["a", "b", "c", "d"]`:
   - ¿A qué evalúa `letras[-1]`?
   - ¿A qué evalúa `letras[:2]`?
   - ¿A qué evalúa `letras[int(int('3' * 2) // 11)]`? ¿Es necesario usar dos veces `int()`?
8. Asuma que `bartulos` contiene la lista `[3.14, "casa", 11, "casa", True]`:
   - ¿A qué evalúa `bartulos.index("casa")`? ¿Por qué?
   - ¿Cómo queda la lista en `bartulos` después de ejecutar `bartulos.append(99)`?
   - ¿Cómo queda la lista en `bartulos` después de ejecutar `bartulos.remove("casa"')` ?¿Por qué?
9. ¿Cuáles son los operadores para la concatenación y la replicación de listas?
10. ¿Cuál es la diferencia entre los métodos `append()` e `insert()` de las listas?
11. ¿Cuáles son dos formas de eliminar valores de una lista?
12. El siguiente bloque de código imprime `['a', 'b', True, 30]`. ¿Por qué?

```
cosas = ["a", "b", True]
bartulos = cosas
bartulos.append(20 + 10)
print(cosas)
```

13. Considere el siguiente bloque de código:

```
marcas = ["Milka", "Cofler", "Águila", "Cadbury", "Lindt", "Toblerone",
"After Eight"]
marcas[2:5]
```

que correctamente muestra `["Águila", "Cadbury", "Lindt"]`. ¿Por qué el siguiente bloque de código no funciona?

```
marcas = ["Milka", "Cofler", "Águila", "Cadbury", "Lindt", "Toblerone",
"After Eight"]
indices = 2:5
marcas[indices]
```

7 Tuplas

1. ¿Por qué la tupla no implementa un método similar al método `.extend()` de las listas?
2. Las tuplas de Python son conocidas por ser inmutables. Por ejemplo, el siguiente bloque de código resulta en un error:

```
tupla = (1, 2, 3)
tupla[1] = 10
```

Sin embargo, el siguiente bloque no arroja ningún error y pareciera que se logra modificar la tupla exitosamente:

```
bartulos = ["Hola", 10, None]
tupla = (1, bartulos, 3)

tupla[1].append("¡Sorpesa!")

print(tupla)
```

¿Qué pasó?

8 Diccionarios

1. ¿Cómo se escribe en código un diccionario vacío?
2. ¿Cómo se ve un diccionario que tiene la clave `"cosa"` y el valor `15`?
3. ¿Cuál es la principal diferencia entre un diccionario y una lista?
4. ¿Qué ocurre si se intenta acceder a `bartulos["cosa"]` cuando `bartulos` es `{"coso": 100}`?
5. Si un diccionario está almacenado en `bartulos`, ¿cuál es la diferencia entre las siguientes expresiones?

```
"cosa" in bartulos
"cosa" in bartulos.keys()
```

6. Suponga el diccionario `datos = {"nombre": "Juan"}`. ¿Por qué la siguiente expresión resulta en `False`?

```
"Juan" in datos
```

9 Control de flujo

9.1 Condicionales

1. Explique qué es una condición y en qué situaciones se utilizaría.
2. Identifique los tres bloques de código en el siguiente ejemplo:

```
codigo = 0

if codigo == 10:
    print("mensaje 1")
    if codigo > 5:
        print("mensaje 2")
    else:
        print("mensaje alternativo")
    print("mensaje final")

print("Fin del programa")
```

   ¿Tienen sentido las comparaciones utilizadas?

3. ¿Cuál es el problema con el siguiente programa? Proponga una solución.

```
numero = 10
if numero < 0:
    print(f"El numero {numero} es negativo")
elif numero < -5:
    print(f"El numero {numero} es menor a -5")
elif numero > 0:
    print(f"El numero {numero} es positivo")
else:
    print(f"El numero {numero} es 0")
```

9.2 Bucles

1. Considere el siguiente programa:

```
for i in range(10):
    print(i)
```

Escriba un programa que realice la misma tarea utilizando un bucle `while`.

2. ¿Cuál es la diferencia entre `range(10)`, `range(0, 10)` y `range(0, 10, 1)` en un bucle `for`?
3. Suponga que `numeros` es una lista que contiene numeros enteros, ¿en qué se diferencian los siguientes bloques de código?

```
for i in numeros:
    if i % 2 == 0:
        break
    print(i)
```

```
for i in numeros:
    if i % 2 == 0:
        continue
    print(i)
```

4. ¿Es posible re-escribir el siguiente bloque de código usando `while True`? ¿Qué modificaciones habría que hacer?

```
suma = 0
i = 0
while suma <= limite:
    suma += numeros[i]
    i += 1
```